

Linux and Docker – Session Documentation

1. Introduction

As part of the technical learning session at ApexaiQ, I learned the basics of **Linux and Docker**. The session mainly focused on understanding the Linux operating system, commonly used Linux commands, Docker and its purpose, creating Docker images, running containers, and port forwarding.

The practical part of the session was useful because instead of only learning the commands theoretically, I was able to execute them in the terminal and observe their output.

2. What is Linux?

Linux is an open-source operating system based on the Linux kernel. It is used to manage the computer's hardware and software resources and provides an interface through which users can interact with the system.

Unlike proprietary operating systems, Linux is **open source**, which means its source code is available and can be modified and distributed.

Linux is commonly used in:

- Servers
- Cloud computing
- Software development
- Networking
- Cybersecurity
- Embedded systems
- DevOps

Some popular Linux distributions are **Ubuntu, Debian, Fedora, and Kali Linux**.

Example

In this session, I used **Ubuntu through WSL (Windows Subsystem for Linux)**. WSL allows Linux commands and tools to be used directly on a Windows system without installing a separate virtual machine.

3. Why do we use Linux?

Linux is widely used in development and server environments because it is lightweight, flexible, secure, and provides a powerful command-line interface.

Some important reasons for using Linux are:

1. Open Source

Linux is freely available and its source code can be modified according to requirements.

2. Command-Line Interface

Linux provides many commands for managing files, folders, processes, networking, and system resources.

3. Stability

Linux systems are known for being stable and are widely used for servers that need to run continuously.

4. Security

Linux provides user permissions and access controls that help protect files and system resources.

5. Development Support

Many programming languages, frameworks, databases, and development tools work very well with Linux.

6. Used in Servers and Cloud

A large number of web servers and cloud systems use Linux because it is efficient and easy to manage remotely.

4. Basic Linux Commands and Their Uses

During the practical session, I used different Linux commands to navigate through the system, create files and folders, and perform basic operations.

Command Use

<code>pwd</code>	Shows the current working directory
<code>ls</code>	Lists files and folders
<code>ls -la</code>	Shows all files including hidden files
<code>cd</code>	Changes the current directory
<code>cd ..</code>	Moves one directory back

<code>mkdir</code>	Creates a new directory
<code>touch</code>	Creates an empty file
<code>cat</code>	Displays file contents
<code>nano</code>	Opens a file for editing
<code>cp</code>	Copies a file or directory
<code>mv</code>	Moves or renames a file
<code>rm</code>	Deletes a file
<code>clear</code>	Clears the terminal screen
<code>whoami</code>	Shows the current logged-in user
<code>history</code>	Shows previously executed commands
<code>exit</code>	Exits the current shell or container

Examples

pwd

`pwd`

It displays the location of the current directory.

Example output:

`/home/prajakta`

ls

`ls`

This command displays the files and folders present in the current directory.

mkdir

`mkdir docker-practice`

This creates a new directory named docker-practice.

cd

`cd docker-practice`

This command moves inside the docker-practice directory.

touch

touch test.txt

This creates an empty file named test.txt.

cat

cat test.txt

This displays the contents of the file.

whoami

whoami

This shows the username of the current user.

clear

clear

This clears the previously displayed output from the terminal.

5. What is Docker?

Docker is a platform used to package, deploy, and run applications in **containers**.

A Docker container contains the application along with the dependencies and environment required to run it. This makes it easier to run the same application on different systems without worrying too much about differences in the environment.

Docker mainly works with:

Dockerfile → Image → Container

- **Dockerfile:** Contains instructions for creating an image.
- **Docker Image:** A packaged, read-only template containing the application and its requirements.
- **Container:** A running instance of a Docker image.

Simple Example

Suppose I have a Python program:

```
print("Hello from Docker")
```

Instead of running the Python program directly on my computer, I can create a Docker image containing the Python environment and my program.

That image can then be used to create a container in which the program runs.

6. Why do we use Docker?

Docker is mainly used to make application deployment easier and more consistent.

1. Same Environment

The application and its dependencies are packaged together, so the application can run in a similar environment on different systems.

2. Easy Deployment

Once an image is created, it can be used to create containers on other systems.

3. Lightweight

Containers generally use fewer resources than traditional virtual machines because they share the host operating system's kernel.

4. Isolation

Applications running in separate containers are isolated from each other.

5. Easy Scaling

Multiple containers can be created from the same image when more instances of an application are required.

6. Useful in DevOps

Docker is commonly used with **CI/CD, cloud computing, microservices, and DevOps workflows**.

7. Docker Image and Container

It is important to understand the difference between an image and a container.

Docker Image

A **Docker image** is a packaged template containing the application, required libraries, dependencies, and instructions needed to run the application.

An image itself is not the running application.

Docker Container

A **container** is a running instance of a Docker image.

A simple way to understand it is:

Docker Image

↓

docker run

↓

Docker Container

For example:

docker run python:3.12

This uses the python:3.12 Docker image to create and run a container.

8. Process of Creating a Docker Image

To create a Docker image for a Python application, the following steps are generally followed:

Create Python Program

↓

Create Dockerfile

↓

Build Docker Image

↓

Run Container

↓

Test Application

Step 1: Create a Python Program

First, create a Python file called app.py.

```
print("Hello from Python inside Docker!")
```

The program can be tested normally using:

```
python3 app.py
```

Expected output

Hello from Python inside Docker!

9. Creating a Dockerfile

A **Dockerfile** is a text file that contains instructions used by Docker to build an image.

Create a file named:

Dockerfile

Use the following code:

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY app.py .
```

```
CMD ["python", "app.py"]
```

Explanation

FROM python:3.12-slim

Specifies the base image. In this case, a lightweight Python 3.12 environment is used.

WORKDIR /app

Creates and sets /app as the working directory inside the image.

COPY app.py .

Copies the Python file from the current computer directory into the Docker image.

CMD ["python", "app.py"]

Specifies the command that should run when the container starts.

10. Building the Docker Image

After creating the Dockerfile, the image can be created using:

```
docker build -t python-demo .
```

Here:

- `docker build` → builds a Docker image.
- `-t` → assigns a name/tag to the image.
- `python-demo` → name of the image.
- `.` → tells Docker to use the current directory as the build context.

After successful execution, check the image using:

```
docker images
```

The `python-demo` image should appear in the list.

11. Running the Docker Container

After creating the image, a container can be created using:

```
docker run python-demo
```

The Docker image is used to create a container and execute the Python program.

Expected output

Hello from Python inside Docker!

This shows that the Python application is successfully running inside the Docker container.

12. Docker Hub and Pushing an Image

Docker Hub is a cloud-based registry where Docker images can be stored and shared.

After creating an image, it can be tagged with a Docker Hub username:

```
docker tag python-demo username/python-demo:latest
```

Here, username should be replaced with the actual Docker Hub username.

Login to Docker Hub:

```
docker login
```

Then push the image:

```
docker push username/python-demo:latest
```

After pushing, the image can be viewed in the Docker Hub repository.

Basic flow

Python Code

↓

Dockerfile



Docker Build



Docker Image



Docker Tag



Docker Login



Docker Push



Docker Hub

13. What is Port Forwarding in Docker?

Port forwarding is used to make an application running inside a Docker container accessible from the host machine.

A container has its own network environment. Therefore, if a web application inside the container is running on a particular port, that port needs to be mapped to a port on the host to access it from the browser.

Docker uses the `-p` option for port mapping.

Syntax

```
docker run -p HOST_PORT:CONTAINER_PORT image_name
```

For example:

```
docker run -p 8080:80 nginx
```

Here:

Host machine port	Container port
-------------------	----------------

8080	→	80
------	---	----

So when we access:

`http://localhost:8080`

the request is forwarded to port 80 inside the Docker container.

Another example

If a Python web application runs on port 8000 inside the container:

```
docker run -p 8000:8000 python-demo
```

The first 8000 is the **host port**, while the second 8000 is the **container port**.

localhost:8000

↓

Host Port 8000

↓

Container Port 8000

↓

Python Application

14. Useful Docker Commands

Command	Purpose
docker --version	Checks Docker version
docker images	Lists Docker images
docker pull image	Downloads an image
docker build	Builds an image
docker run	Creates and runs a container
docker ps	Shows running containers
docker ps -a	Shows all containers
docker start	Starts a stopped container
docker stop	Stops a running container
docker restart	Restarts a container
docker exec	Executes a command inside a running container
docker logs	Displays container logs

docker rm	Removes a container
docker rmi	Removes an image
docker pull	Downloads an image from a registry
docker push	Uploads an image to a registry
docker login	Logs into a Docker registry

15. Difference Between Linux and Docker

Linux and Docker are related but they are not the same thing.

Linux	Docker
Linux is an operating system/kernel-based platform	Docker is a containerization platform
Manages system resources and hardware	Manages containers and application environments
Provides commands and system tools	Provides commands to create and manage containers
Can run applications directly	Runs applications inside containers
Commonly used as a server OS	Commonly used for application deployment

Docker containers on Linux use the Linux kernel features provided by the host system.